

Proyecto 1: Reproductor de música

Matías Yevenes

Carlos Cofré

Eban Delgado

Estructura de Datos
2 de octubre de 2026

Docente: Jacqueline Aldridge

Informe técnico del proyecto

Índice

1. Introducción	3
2. Objetivos	3
3. Estructuras y catálogo	3
3.1. Generación y validación	3
4. Algoritmos	3
4.1. Ordenamiento iterativo	3
4.2. Ordenamiento recursivo	4
4.3. Búsqueda binaria recursiva	4
5. Consultas y estadísticas	4
6. Reproducción, cola e historial	4
7. Persistencia y ejecución	4
8. Verificación	5
9. Aportes asignados	5
10.Diferencias y alcance	5
11.Declaración de uso de inteligencia artificial	6

1. Introducción

El proyecto implementa una simulación de un reproductor de música. Mantiene un catálogo de canciones en arreglos, permite ordenar y consultar sus datos, simula la reproducción, administra una cola y conserva un historial reciente. No reproduce archivos de audio: el temporizador de Allegro representa el avance de una canción y actualiza su contador al finalizar.

El desarrollo sigue los requisitos del enunciado *Proyecto 1: Reproductor de música* de la asignatura Estructura de Datos. La interfaz de uso activa es una ventana gráfica hecha con Allegro 5.

2. Objetivos

- Organizar un catálogo de canciones mediante arreglos estáticos.
- Aplicar ordenamiento iterativo y recursivo, además de búsqueda binaria recursiva.
- Consultar canciones, artistas, géneros y estadísticas de reproducción.
- Simular reproducción, cola e historial, manteniendo los datos numéricos dentro de rangos válidos.
- Cargar y exportar el catálogo en formato CSV.

3. Estructuras y catálogo

La estructura **Songs** contiene ID, duración, año, reproducciones y cadenas para título, artista, género y álbum. **Album** agrupa una matriz de **Songs**, más los arreglos y contadores de cola e historial. La estructura **ReferenciaCancion** almacena dos índices (álbum y canción); se ordenan estas referencias en vez de mover canciones, por lo que las operaciones conservan intactos los registros.

El tamaño actual es de 15 álbumes con 25 canciones por álbum, para un total de 375 registros. La memoria se reserva mediante arreglos estáticos; no se utiliza asignación dinámica.

3.1. Generación y validación

Generar.c asigna IDs consecutivos desde 1 hasta 375. Después mezcla las canciones dentro de cada álbum mediante Fisher–Yates, conservando juntas las canciones que pertenecen a cada álbum. La duración generada está entre 120 y 219 segundos, el año entre 2000 y 2008 y las reproducciones entre 0 y 999 999. Los textos aleatorios tienen diez letras minúsculas; cada álbum conserva el mismo nombre en sus 25 canciones.

Al cargar el CSV se comprueba que los campos estén presentes, que la duración sea positiva, que las reproducciones no sean negativas, que el año esté entre 1900 y 2100 y que los IDs sean únicos y estén en el rango esperado. Si el archivo no se puede cargar o no cumple estas condiciones, se genera un catálogo nuevo y se guarda.

4. Algoritmos

4.1. Ordenamiento iterativo

ordenar_referencias_burbuja implementa Bubble Sort. Compara canciones mediante la referencia a sus posiciones y permite ordenar por ID, año, duración, reproducciones, género, artista, título o álbum. En la ventana, primero se elige un álbum y luego se ordenan sus canciones.

4.2. Ordenamiento recursivo

`ordenar_referencias_recursivo` implementa Quick Sort con partición. Se utiliza para ordenar el índice del catálogo por ID antes de la búsqueda binaria y para construir el ranking descendente de reproducciones.

4.3. Búsqueda binaria recursiva

`buscar_id_binaria_recursiva` busca un ID en el índice ordenado. En cada llamada compara el ID buscado con el elemento central y continúa únicamente en la mitad correspondiente. Devuelve la posición del índice o -1 si no existe.

La búsqueda por título, artista y género admite coincidencias parciales y no distingue mayúsculas de minúsculas; esas consultas recorren el catálogo linealmente. La búsqueda binaria se aplica al ID, que es el campo numérico único usado para construir el índice.

5. Consultas y estadísticas

La interfaz permite buscar canciones por ID, título, artista o género. La vista de estadísticas permite listar artistas disponibles, listar y contar coincidencias por género, mostrar el top N de canciones por reproducciones y encontrar la canción o canciones con más reproducciones dentro de un artista o género indicado.

El listado de artistas conserva una referencia a una canción por cada nombre de artista distinto. Los resultados se muestran con sus campos principales y se pueden seleccionar para abrir la vista de reproducción.

6. Reproducción, cola e historial

La duración se simula con un temporizador Allegro. Al terminar una canción se incrementa `n_reproducciones`; si hay canciones pendientes, se retira y reproduce el primer elemento de la cola.

La cola admite hasta diez canciones. Cada elemento se representa mediante los índices de álbum y canción empaquetados en un entero. La inserción se realiza al comienzo, se rechazan duplicados y se puede quitar el elemento seleccionado, quitar por ID, vaciar la cola o reproducir el primero. El historial también conserva hasta diez elementos, colocando el más reciente al inicio y desplazando los anteriores al superar el límite.

7. Persistencia y ejecución

El archivo `catalogo.csv` se carga al iniciar y se actualiza al cerrar el programa. Además, al finalizar se genera `catalogo_actualizado.csv` con los datos actuales, incluidos los contadores de reproducción.

La compilación en Linux utiliza el `Makefile` y las dependencias de Allegro 5 obtenidas con `pkg-config`:

```
make
make run
make clean
```

8. Verificación

La compilación limpia se realizó con `make clean && make`, usando `-Wall -Wextra -std=c99`, y finalizó sin errores ni advertencias. La revisión de ambos CSV comprobó 375 registros, IDs únicos dentro del rango y valores numéricos válidos.

No se incorporó un conjunto automatizado de pruebas al repositorio; la prueba de algoritmos fue una verificación puntual durante el desarrollo.

9. Aportes asignados

La distribución concreta sigue las responsabilidades generales descritas en el README del proyecto y asigna grupos identificables de funciones a cada integrante. Los mismos responsables están señalados mediante comentarios en los archivos fuente.

Integrante	Responsabilidad asignada	Secciones del código
Matías Yevenes	Diseño e interacción de la ventana Allegro: navegación, controles, presentación de álbumes, canciones, reproductor, búsqueda, estadísticas, cola e historial.	<code>main.c</code> : dibujo de vistas, controles, eventos de ratón y teclado e inicialización de Allegro.
Carlos Cofré	Algoritmos y consultas: comparación y ordenamiento de referencias, búsqueda binaria por ID, búsqueda por campos y cálculo de estadísticas.	<code>Lista_Albumes.c</code> ; funciones de búsqueda, preparación de resultados y estadísticas en <code>main.c</code> .
Eban Delgado	Funcionalidad de datos y reproducción: estructuras, generación, validación y persistencia CSV, cola, historial y avance simulado.	<code>Reproductor.h</code> , <code>Generar.c</code> , <code>Fila_de_Reproduccion.c</code> ; carga/exportación y ciclo de reproducción en <code>main.c</code> .

10. Diferencias y alcance

El enunciado solicita un menú de línea de comandos; la versión actual utiliza Allegro como interfaz principal. El ordenamiento interactivo se aplica a las canciones del álbum seleccionado, no a una vista global del catálogo. El criterio por álbum queda disponible, aunque dentro de un álbum todos sus registros comparten ese valor. Los nombres de artista y género se generan como cadenas aleatorias, en lugar de seleccionarse de listas predefinidas. Estas diferencias se dejan registradas para no presentar como implementadas funcionalidades distintas de las que ofrece el código.

11. Declaración de uso de inteligencia artificial

Durante la preparación de esta versión se utilizó GitHub Copilot como apoyo para revisar algoritmos e integrar la interfaz como diseño (en Allegro). La asistencia generó o modificó secciones de `Fila_de_Reproduccion.c`, `main.c`, y `Reproductor.h`; los cambios se compilaron y se verificaron según se describe en la sección de pruebas.

El uso personal de IA se indica a continuación:

- **Matías Yevenes:** *[GEMINI: Apoyo en el diseño de la interfaz, hecha en Allegro en C y apoyo en el ordenamiento numérico y alfabético de los nombres de las canciones / duraciones de estas, etc. Para apoyo básico en el uso de ordenamiento de datos.]*
- **Carlos Cofré:** *[COPILOT: Uso de Inteligencia artificial en la verificación de código para revisión de redundancia, indentación, y orden, en general.]*
- **Eban Delgado:** *[COPILOT: Uso de Inteligencia artificial para revisión de implementación en torno a lo pedido en el enunciado y para implementación de la generacion de csr para el proyecto.]*

Funciones Generadas con Inteligencia artificial utilizadas en el código:

```
en "main.c" : "int main(void), typedef struct {} estado_ventana; & static int  
    punto_en_rectangulo(int x, int y, int rx, int ry, int rw, int rh)"
```

Referencias

- [1] Aldridge, Jacqueline. *Proyecto 1: Reproductor de música*. Curso Estructura de Datos, 2026. Enunciado proporcionado como `EDA_Tarea1.pdf`.